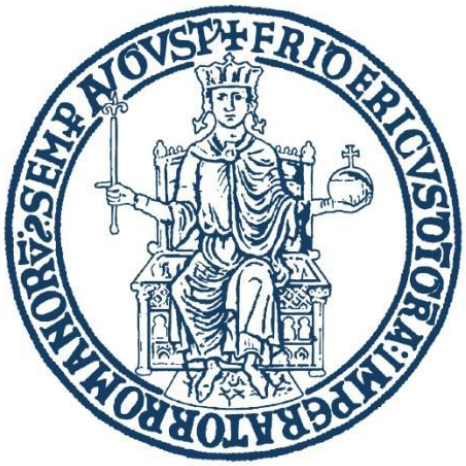


UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

DIPARTIMENTO DI INFORMATICA

Basi di Dati I

Anno Accademico 2024/25

Analisi, Progettazione e Sviluppo di una BASE DI DATI

Autore: Antonio Andrea Montella [N86005652]

Docente: Silvio Barra

Traccia 3

Sommario

1. Introduzione 3
1.1. Descrizione del Progetto 3
2. Progettazione Concettuale 4
2.1. Class Diagram 5
2.2. Ristrutturazione Class Diagram 6
2.2.1. Analisi delle classi con singolo attributo e delle Ridondanze 6
2.2.2. Analisi degli attributi Strutturati 6
2.2.3. Analisi degli attributi Multi-Value 7
2.2.4. Analisi delle gerarchie di Specializzazione 7
2.2.5. Scelta degli Identificatori Principali 8
2.3. Class Diagram Ristrutturato 9
2.4. Dizionario delle Classi 10
2.5. Dizionario delle Associazioni 12
2.6. Dizionario dei Vincoli 13
3. Progettazione Logica 14
3.1. Schema Logico 14
3.2. Tabelle di Supporto Storico (Audit) 15
4. Progettazione Fisica 17
4.1. Definizione Tabelle 17
4.1.1. Definizione domini ENUM 17
4.1.2. Definizione tabella UTENTI 17
4.1.3. Definizione tabella VOLI 18
4.1.4. Definizione tabella PRENOTAZIONI 18
4.1.5. Definizione tabella TICKETS 19
4.1.6. Definizione tabella AUDIT *(log_cancellazioni_tickets)* 19
4.2. Implementazione Vincoli/Trigger 20
4.2.1. Implementazione del vincolo sulla Data del Volo 20
4.2.2. Implementazione del vincolo di Prenotazione 21
4.2.3. Aggiornamento automatico Stato (in base a ritardo) 21
4.2.4. Propagazione a cascata delle Cancellazioni 22
4.2.5. Implementazione Storico tramite Audit Log 23
4.3. Implementazione Viste (Views) 24
4.3.1. Vista delle Prenotazioni Complete 24
4.3.2. Vista dei Ticket Completi 24

1. Introduzione

L'elaborato ha lo scopo di documentare la progettazione e lo sviluppo di una base di dati relazionale, avvalendosi del DMBS PostgreSQL. Il progetto, svolto dal solo studente Montella Antonio Andrea del corso di laurea in Informatica presso l'Università degli Studi di Napoli Federico II.

1.1. Descrizione del Progetto

Il progetto è stato sviluppato seguendo meticolosamente le richieste presentate nella traccia numero 3 presentata durante il corso di Basi di Dati. L'obiettivo è quello di gestire e garantire le operazioni principali di un aeroporto, nello specifico quello di Napoli.

Il sistema permette di amministrare la programmazione dei voli (sia quelli in partenza che quelli in arrivo), l'intero flusso relativo alla prenotazione dei voli e alla conseguente emissione dei tickets. Questo comporta quindi anche la registrazione e l'autenticazione degli utenti. Inoltre, l'applicativo offre funzionalità aggiuntive dedicate agli amministratori come quella dell'aggiornamento dello stato dei voli, la gestione dei ritardi, cancellazioni e l'assegnazione dei gate d'imbarco per i voli in partenza. Così facendo si garantisce la coerenza dei dati e l'automazione dei processi tramite appositi trigger, i quali verranno presentati in seguito.

Verranno riportate, passo dopo passo, tutte le fasi della progettazione e dello sviluppo dell'intero "mini-mondo".

2. Progettazione Concettuale

L'analisi dei requisiti è il punto iniziale attraverso il quale verranno create le fondamenta del progetto. È un passaggio essenziale per poter formulare una prima idea sulle modalità di sviluppo logico del database, rimanendo quindi su un alto livello di astrazione per il momento.

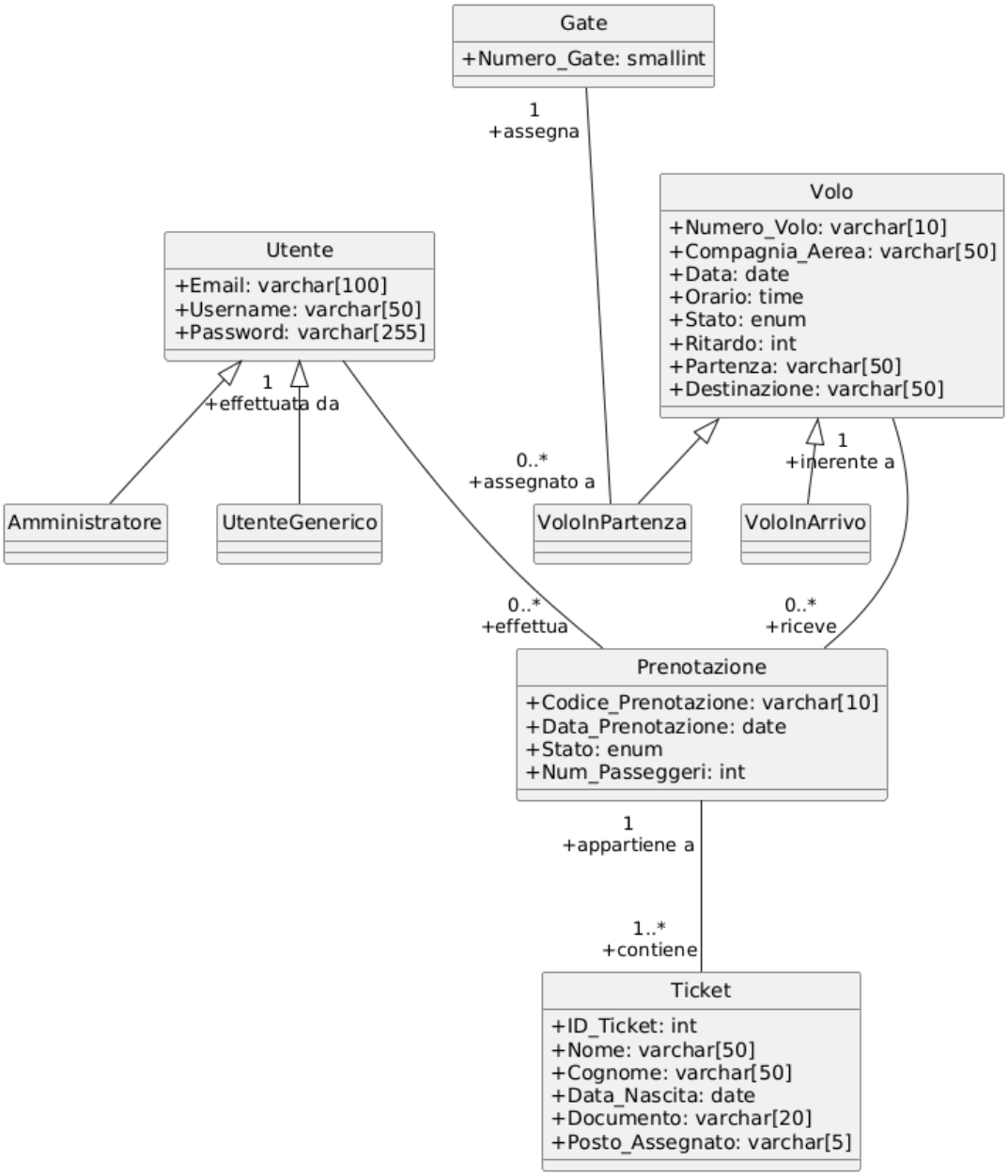
La struttura iniziale si basa su due gerarchie principali di ereditarietà.

- **Utente:** Prevede una superclasse Utente *(padre)* da cui derivano due sottoclassi *(figli)*: **Amministratore** e **UtenteGenerico**. La generalizzazione è totale *(quindi un utente è Admin o UtenteGenerico, non può essere entrambi, e non può essere diverso da uno dei due!)*.
- **Volo:** Prevede una superclasse Volo, da cui derivano le sottoclassi **VoloInPartenza** e **VoloInArrivo**. Anche questa generalizzazione è totale.

A queste seguono le classi **Prenotazione**, **Ticket** e **Gate**. I primi due gestiscono la logica di business legata all'acquisto dei biglietti, mentre il terzo viene delegato agli amministratori per gestire la zona di raccolta dei passeggeri di uno specifico volo.

Il modello concettuale dell'applicativo è stato progettato secondo il paradigma UML. Questa rappresentazione di Class Diagram consente la descrizione della struttura concettuale di una base di dati, mostrando le classi, i loro attributi, le associazioni tra esse.

2.1. Class Diagram



2.2. Ristrutturazione Class Diagram

Il processo di ristrutturazione del Class Diagram ha il fine di rendere quest'ultimo idoneo alla traduzione in schemi relazionali e di migliorarne l'efficienza. Questo procedimento si basa sui seguenti punti:

- *Analisi delle ridondanze*
- *Analisi degli attributi strutturati*
- *Analisi degli attributi multi-value*
- *Analisi delle gerarchie di specializzazione*
- *Scelta degli identificatori principali*

2.2.1. Analisi delle classi con singolo attributo e delle Ridondanze

La classe **Gate**, introdotta concettualmente per rappresentare il luogo fisico dell'aeroporto, risulta possedere il solo attributo identificativo Numero Gate. La creazione di una tabella dedicata avrebbe introdotto una ridondanza strutturale e un dispendio inutile in termini di operazioni di Join. Pertanto, si è proceduto all'eliminazione dell'entità Gate e la sua associazione (*assegnato a*) è stata assorbita dalla classe Volo proprio tramite l'attributo (opzionale) Gate di Imbarco. Non sono state riscontrate altre ridondanze o attributi derivati il cui mantenimento avrebbe causato anomalie.

2.2.2. Analisi degli attributi Strutturati

Gli attributi strutturati (o composti) devono essere "trasformati" in attributi semplici prima della traduzione logica. In questo schema concettuale tutti gli attributi individuati (come Nome, Cognome, Data, Orario) risultano già nativamente **atomici**! Pertanto, non si è resa necessaria alcuna operazione di scomposizione.

2.2.3. Analisi degli attributi Multi-Value

Nel modello non sono presenti attributi *multi-value* o *composti*. Lo **Stato** del volo viene aggiornato dinamicamente, ma non è strettamente derivato da calcoli su altri campi, in quanto dipende sia dall'orario che da interventi esterni (ad esempio una cancellazione manuale o l'inserimenti di un **Ritardo** da parte di un amministratore). Pertanto, non si segnalano *ridondanze* da eliminare in questa fase.

2.2.4. Analisi delle gerarchie di Specializzazione

Il modello concettuale originale presenta due gerarchie di generalizzazione, entrambe **totali**. Per la traduzione si è scelto l'approccio del "collasso verso l'alto" (collasso i figli nel padre, *Single Table Strategy*) per evitare l'eccessiva frammentazione dei dati e ridurre il costo delle operazioni di Join.

- **Gerarchia Utente (Amministratore / Utente Generico):** Le entità figlie non presentano attributi aggiuntivi né associazioni specifiche. Per questo motivo, la gerarchia è stata collassata nella superclasse Utente. È stato introdotto l'attributo discriminatore booleano *is_admin* per differenziare logicamente i privilegi degli utenti all'interno della stessa classe.
- **Gerarchia Volo (Volo in Partenza / Volo in Arrivo):** Analogamente, la gerarchia dei voli è stata collassata verso l'alto nella superclasse Volo. L'unico attributo specifico della classe figlia *Volo in Partenza*, ovvero Gate di Imbarco, è stato "tirato su" nella classe padre. Questo attributo accetterà valori NULL nel caso in cui il record rappresenti un volo in arrivo. È stato aggiunto l'attributo discriminatore Tipo Volo per indicare la natura del volo ('PARTENZA' o 'ARRIVO').

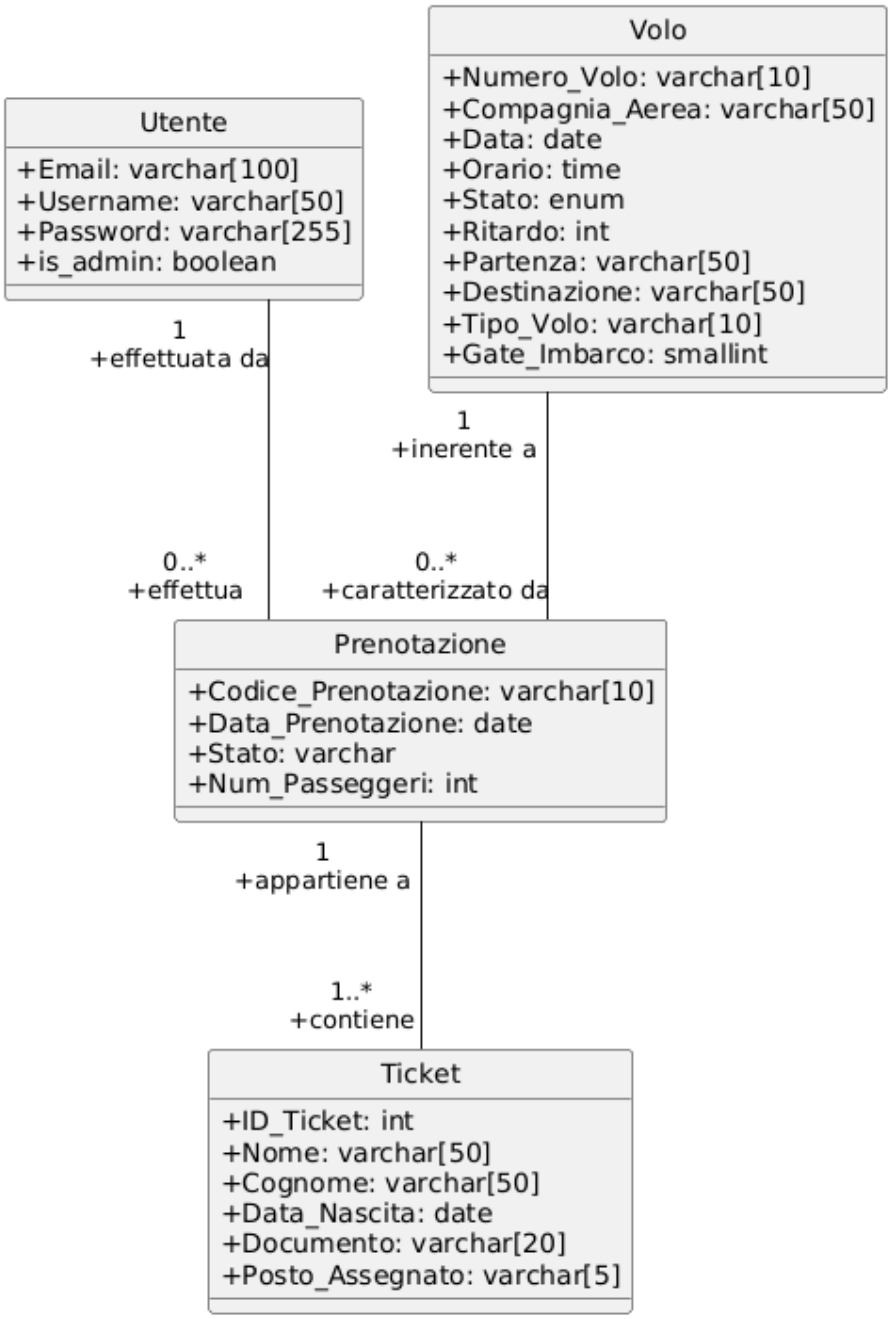
2.2.5. Scelta degli Identificatori Principali

Per ogni classe del modello è stato individuato un identificatore univoco (chiave primaria):

- **Utente:** si è scelto l'attributo **Email** come identificatore univoco per l'accesso. Si sarebbe potuto utilizzare un identificatore surrogato (ad esempio *ID_Utente* magari *auto-incrementale*), però ho preferito utilizzare **Email** per semplicità del mini-mondo in analisi, rimanendo almeno dove possibile con chiavi naturali, seppur non tenendo conto della "best practice". Anche **username** sarebbe una chiave candidata, ma qui ho preferito aggiungere solamente il vincolo di unicità (*constraint utenti_username_key*), così da poter implementare in futuro una banale funzione di "*aggiorna_username*".
- **Volo:** la stringa alfanumerica **Numero Volo** identifica univocamente ogni volo (es. FR688).
- **Prenotazione:** viene generato un **Codice Prenotazione** alfanumerico univoco progressivo (es. P20001).
- **Ticket:** viene utilizzato un **ID Ticket** numerico auto-incrementale.

2.3. Class Diagram Ristrutturato

Sia nel primo formalismo che nel Class Diagram ristrutturato, viene omessa l'aggiunta delle due "classi" enumeration. Questo perché è stato preferito focalizzare l'attenzione sulle entità reali, sebbene la loro aggiunta almeno nel Class Diagram precedente alla ristrutturazione non fosse un vero e proprio errore (come visto ad inizio corso).



2.4. Dizionario delle Classi

Classe	Descrizione	Attributi
Utente	Descrive ciascun utente iscritto al sistema gestionale, raggruppando al suo interno sia i clienti generici che gli amministratori.	• **Email** (varchar): Indirizzo e-mail dell'utente. • **Username** (varchar): Nome utente univoco. • **Password** (varchar): Credenziale di accesso. • **is_admin** (boolean): Attributo discriminatore che indica se l'utente possiede i privilegi di amministratore.
Volo	Descrive i singoli voli gestiti dall'aeroporto (sia quelli in partenza che quelli in arrivo).	• **Numero_Volo** (varchar): Identificativo univoco alfanumerico. • **Compagnia_Aerea** (varchar): Compagnia che opera il volo. • **Data** (date): Data in cui il volo è schedulato. • **Orario** (time): Orario di partenza o arrivo previsto. • **Stato** (enum): Stato attuale (es. Programmato, Cancellato). • **Ritardo** (int): Ritardo accumulato in minuti. • **Partenza** (varchar): Città di partenza. • **Destinazione** (varchar): Città di destinazione. • **Tipo_Volo** (varchar): Attributo discriminatore ('PARTENZA' o 'ARRIVO').

		• **Gate_Imbarco** (smallint, opzionale): Gate assegnato (valorizzato solo per i voli in partenza).
Prenotazione	Descrive l'atto di acquisto/prenotazione effettuato da un utente per un determinato volo.	• **Codice_Prenotazione** (varchar): Codice alfanumerico univoco. • **Data_Prenotazione** (date): Data di emissione. • **Stato** (enum): Stato della transazione (es. Confermata, Cancellata). • **Num_Passeggeri** (int): Numero di posti riservati in questa prenotazione.
Ticket	Descrive il singolo biglietto nominativo assegnato a un passeggero all'interno di una prenotazione.	• **ID_Ticket** (int): Identificativo univoco surrogato (auto-incrementale). • **Nome** (varchar): Nome del passeggero. • **Cognome** (varchar): Cognome del passeggero. • **Data_Nascita** (date): Data di nascita del passeggero. • **Documento** (varchar): Identificativo del documento d'identità. • **Posto_Assegnato** (varchar): Codice alfanumerico del sedile (es. A1).

2.5. Dizionario delle Associazioni

Nome	Descrizione	Classi coinvolte
Effettua Prenotazione	Esprime la relazione tra un Utente e le Prenotazioni da lui generate nel sistema.	**Utente [1]** ruolo *effettuata da*: indica l'Utente che ha generato una determinata Prenotazione. **Prenotazione [0..*]** ruolo *effettua*: indica l'insieme di Prenotazioni effettuate da uno specifico Utente.
Prenotazione del Volo	Esprime la relazione tra un Volo e l'insieme delle Prenotazioni ad esso associate.	**Volo [1]** ruolo *inerente a*: indica il Volo a cui una determinata Prenotazione fa riferimento. **Prenotazione [0..*]** ruolo *caratterizzato da*: indica l'insieme di Prenotazioni ricevute da uno specifico Volo.
Ticket Prenotazione	Esprime la composizione di una Prenotazione, suddivisa nei singoli biglietti (Ticket) per ciascun passeggero.	**Prenotazione [1]** ruolo *appartiene a*: indica la Prenotazione principale di cui fa parte un determinato Ticket. **Ticket [1..*]** ruolo *contiene*: indica l'insieme dei Ticket passeggeri contenuti all'interno di una determinata Prenotazione.

2.6. Dizionario dei Vincoli

Nome	Descrizione
Esistenza Utente	L'utente che effettua una prenotazione deve obbligatoriamente essere registrato ed esistere nel sistema. Se l'utente viene eliminato, le sue prenotazioni decadono.
Esistenza Volo	Una prenotazione deve essere sempre associata ad un volo esistente e programmato. L'eliminazione di un volo comporta la cancellazione a cascata delle relative prenotazioni.
Integrità Ticket	Un ticket passeggero deve necessariamente appartenere ad una prenotazione valida.
Limite Passeggeri	Il numero di passeggeri per singola prenotazione deve essere strettamente maggiore di 0 e non può superare il limite massimo di 9 persone.
Validità Gate	Il gate d'imbarco (se assegnato a un volo in partenza) deve corrispondere a un numero fisico reale, compreso tra 1 e 20.
Consistenza Ritardo	Il ritardo accumulato da un volo, espresso in minuti, non può mai assumere un valore negativo.
Consistenza Tipo Volo	Un volo deve essere esplicitamente classificato o come "Partenza" o come "Arrivo", non sono ammessi stati intermedi.
Unicità Username	All'interno del sistema non possono esistere due utenti registrati che utilizzano lo stesso Nome Utente (Username).
Unicità Posto Passeggero	All'interno della medesima prenotazione, non è consentito assegnare lo stesso identificativo di posto a sedere (es. A1) a due passeggeri differenti.

3. Progettazione Logica

In questa fase si scenderà ad un livello di astrazione più basso rispetto al precedente. Lo schema concettuale ristrutturato verrà ora tradotto in uno schema logico, dipendente dalla struttura dei dati prescelta (relazionale pura).

3.1. Schema Logico

Di seguito è riportato lo schema logico della base di dati. Al suo interno le chiavi primarie (PK) sono indicate con una sottolineatura, mentre i vincoli di integrità referenziale (chiavi esterne) sono esplicitati subito sotto la definizione della rispettiva relazione (oltre ad un asterisco, così da individuarli subito).

- **Utenti** (*email*, username, password, is_admin)
- **Voli** (*numero_volo*, compagnia_aerea, data_volo, orario_previsto, stato, ritardo, partenza, destinazione, tipo_volo, gate_imbarco)
- **Prenotazioni** (*codice_prenotazione*, email, numero_volo, data_prenotazione, stato_prenotazione, numero_passeggeri)
    - email -> Utenti.email
    - numero_volo -> Voli.numero_volo
- **Tickets** (*id_ticket*, codice_prenotazione, nome, cognome, numero_documento, data_nascita, posto_assegnato)
    - codice_prenotazione -> Prenotazioni.codice_prenotazione

3.2. Tabelle di Supporto Storico (Audit)

Nel passaggio dalla progettazione concettuale allo schema logico (e successivamente a quello fisico), è stata preferita come scelta l'introduzione di una tabella aggiuntiva non presente nel Class Diagram originale, denominata *log_cancellazioni_tickets*.

Questa tabella non modella un'entità del dominio (non rappresenta un oggetto dell'aeroporto), bensì risponde a uno specifico requisito tecnico e di sicurezza, ovvero l'**Auditing**. Nei sistemi reali, l'eliminazione fisica di un record (es. un passeggero rimosso o un volo cancellato) comporta la perdita irreversibile del dato. Per tutelare il sistema e garantire un livello adeguato di assistenza clienti, si è deciso di implementare un meccanismo di storicizzazione.

Questa parte non è stata trattata nel corso, e non era prevista come parte integrante dello sviluppo progettuale. Nonostante ciò, ho preferito introdurla proprio dopo essermi chiesto come avrebbe fatto l'assistenza clienti a "riorganizzare" la prenotazione tramite un altro volo (o ad esempio l'emissione di un ticket sostitutivo, rimborso, ...), così da soddisfare le possibili esigenze di un'utente generico finale. Questo aspetto forse sarebbe più appropriato farlo gestire direttamente dalla compagnia aerea in questione, ma per rendere l'idea ho preferito trattarlo senza entrare troppo nel dettaglio della realtà in questione.

La tabella **log_cancellazioni_tickets** presenta i seguenti attributi:

- **id_log** (Chiave Primaria auto-incrementale)
- **id_ticket_originale** (Intero)
- **nome, cognome, numero_documento** (Dati anagrafici)
- **codice_prenotazione** (Riferimento testuale)
- **data_cancellazione** (Timestamp generato al momento dell'eliminazione)

Caratteristiche progettuali: A differenza delle altre tabelle, questa relazione è volutamente sprovvista di vincoli di chiave esterna (Foreign Key) verso le tabelle principali. Questa scelta è fondamentale! Infatti, se il log avesse una chiave esterna verso la prenotazione, non potremmo eliminare la prenotazione originale senza prima eliminare anche il log, vanificando lo scopo stesso dello storico. Inoltre, questa tabella non viene mai popolata manualmente dall'applicativo, ma è gestita in totale autonomia dal **DBMS** tramite l'ausilio di apposite **Regole Attive (Trigger)** che intercettano gli eventi (in questo caso di tipo DELETE), del quale analizzeremo i dettagli più avanti.

4. Progettazione Fisica

In questo capitolo verrà riportata l'implementazione dello schema logico precedentemente descritto all'interno del DBMS relazionale PostgreSQL.

4.1. Definizione Tabelle

Di seguito sono riportate le definizioni delle tabelle, la creazione dei tipi di dato personalizzati (ENUM) e l'esplicitazione dei vincoli intra-relazionali e inter-relazionali di base.

4.1.1. Definizione domini ENUM

```sql
CREATE TYPE stato_prenotazione AS ENUM (
    'CONFERMATA',
    'IN_ATTESA',
    'CANCELLATA'
);

CREATE TYPE stato_volo AS ENUM (
    'PROGRAMMATO',
    'DECOLLATO',
    'IN_RITARDO',
    'ATTERRATO',
    'CANCELLATO'
);
```

4.1.2. Definizione tabella UTENTI

```sql
CREATE TABLE utenti (
    email VARCHAR(255) PRIMARY KEY,
    username VARCHAR(25) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    is_admin BOOLEAN NOT NULL
);

```

4.1.3. Definizione tabella VOLI

```sql
CREATE TABLE voli (
    numero_volo VARCHAR(50) PRIMARY KEY,
    compagnia_aerea VARCHAR(100) NOT NULL,
    partenza VARCHAR(100) NOT NULL,
    destinazione VARCHAR(100) NOT NULL,
    data_volo DATE NOT NULL,
    orario_previsto TIME NOT NULL,
    stato stato_volo NOT NULL,
    ritardo INTEGER DEFAULT 0,
    tipo_volo VARCHAR(20) NOT NULL,
    gate_imbarco SMALLINT,

    CONSTRAINT chk_range_gate CHECK (gate_imbarco IS NULL
                                     OR (gate_imbarco > 0 AND gate_imbarco < 21)),
    CONSTRAINT chk_ritardo CHECK (ritardo >= 0),
    CONSTRAINT chk_tipo_volo CHECK (tipo_volo IN ('PARTENZA', 'ARRIVO'))
);
```

4.1.4. Definizione tabella PRENOTAZIONI

```sql
CREATE TABLE prenotazioni (
    codice_prenotazione VARCHAR(50) PRIMARY KEY,
    email VARCHAR(255) NOT NULL,
    numero_volo VARCHAR(50) NOT NULL,
    stato_prenotazione stato_prenotazione DEFAULT 'IN_ATTESA',
    numero_passeggeri INTEGER NOT NULL DEFAULT 1,
    data_prenotazione TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT prenotazioni_email_fkey FOREIGN KEY (email)
        REFERENCES utenti(email) ON DELETE CASCADE,
    CONSTRAINT prenotazioni_numero_volo_fkey FOREIGN KEY (numero_volo)
        REFERENCES voli(numero_volo) ON DELETE CASCADE,
    CONSTRAINT chk_numero_passeggeri CHECK (numero_passeggeri > 0
                                            AND numero_passeggeri <= 9)
);
```

4.1.5. Definizione tabella TICKETS

```sql
CREATE SEQUENCE tickets_id_ticket_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE tickets (
    id_ticket INTEGER PRIMARY KEY DEFAULT nextval('tickets_id_ticket_seq'),
    nome VARCHAR(100) NOT NULL,
    cognome VARCHAR(100) NOT NULL,
    numero_documento VARCHAR(100) NOT NULL,
    data_nascita VARCHAR(50) NOT NULL,
    posto_assegnato VARCHAR(10) NOT NULL,
    codice_prenotazione VARCHAR(50) NOT NULL,

    CONSTRAINT tickets_codice_prenotazione_fkey FOREIGN KEY (codice_prenotazione)
        REFERENCES prenotazioni(codice_prenotazione) ON DELETE CASCADE,
    CONSTRAINT posto_prenotazione_unici UNIQUE (posto_assegnato, codice_prenotazione)
);
```

4.1.6. Definizione tabella AUDIT *(log_cancellazioni_tickets)*

```sql
CREATE SEQUENCE log_cancellazioni_tickets_id_log_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE log_cancellazioni_tickets (
    id_log INTEGER PRIMARY KEY DEFAULT nextval('log_cancellazioni_tickets_id_log_seq'),
    id_ticket_originale INTEGER,
    codice_prenotazione VARCHAR(50),
    nome_passeggero VARCHAR(100),
    cognome_passeggero VARCHAR(100),
    data_cancellazione TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    motivo VARCHAR(255)
);
```

4.2. Implementazione Vincoli/Trigger

Di seguito sono riportate le implementazioni dei vincoli complessi e delle regole attive (**trigger**) che garantiscono la consistenza dei dati e **automatizzano** i processi di business dell'aeroporto, che non era possibile gestire tramite semplici vincoli intra-relazionali.

4.2.1. Implementazione del vincolo sulla Data del Volo

Un volo non può essere riprogrammato o inserito con una data antecedente a quella odierna.

```
CREATE OR REPLACE FUNCTION check_data_volo_valida()
RETURNS trigger AS $$
BEGIN
    IF NEW.data_volo < CURRENT_DATE THEN
        RAISE EXCEPTION 'Errore: Non è possibile programmare un volo
                            nel passato (Data inserita: %)', NEW.data_volo;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_data_volo
BEFORE INSERT OR UPDATE OF data_volo ON voli
FOR EACH ROW EXECUTE FUNCTION check_data_volo_valida();
```

4.2.2. Implementazione del vincolo di Prenotazione

Un utente può effettuare una prenotazione solo se il volo di riferimento si trova in uno stato idoneo (*Programmato* o *In Ritardo*).

```
CREATE OR REPLACE FUNCTION check_stato_volo_prenotabile()
RETURNS trigger AS $$
DECLARE
    stato_attuale stato_volo;
BEGIN
    SELECT stato INTO stato_attuale FROM voli WHERE numero_volo = NEW.numero_volo;
    IF stato_attuale NOT IN ('PROGRAMMATO', 'IN_RITARDO') THEN
        RAISE EXCEPTION 'Impossibile prenotare: il volo è %', stato_attuale;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_prenotazione_valida
BEFORE INSERT ON prenotazioni
FOR EACH ROW EXECUTE FUNCTION check_stato_volo_prenotabile();
```

4.2.3. Aggiornamento automatico Stato (in base a ritardo)

Se un amministratore inserisce un ritardo maggiore di o per un volo 'Programmato', il suo stato passa automaticamente a *In Ritardo*. Viceversa, se il ritardo viene azzerato, lo stato torna a *Programmato*.

```
CREATE OR REPLACE FUNCTION aggiorna_stato_ritardo()
RETURNS trigger AS $$
BEGIN
    -- Se viene inserito un ritardo maggiore di 0 e il volo era Programmato
    IF NEW.ritardo > 0 AND OLD.ritardo = 0 AND NEW.stato = 'PROGRAMMATO' THEN
        NEW.stato = 'IN_RITARDO';
    -- Se il ritardo viene azzerato e il volo era In Ritardo
    ELSIF NEW.ritardo = 0 AND OLD.ritardo > 0 AND NEW.stato = 'IN_RITARDO' THEN
        NEW.stato = 'PROGRAMMATO';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_aggiorna_stato_ritardo
BEFORE UPDATE OF ritardo ON voli
FOR EACH ROW EXECUTE FUNCTION aggiorna_stato_ritardo();
```

4.2.4. Propagazione a cascata delle Cancellazioni

Se un volo viene cancellato, tutte le relative prenotazioni passano allo stato *CANCELLATA*. A sua volta, quando una prenotazione viene cancellata, i biglietti ad essa associati vengono rimossi fisicamente per "liberare" ipoteticamente i posti sul velivolo.

```sql
-- 1. Propagazione dal Volo alla Prenotazione
CREATE OR REPLACE FUNCTION propaga_cancellazione_volo()
RETURNS trigger AS $$
BEGIN
    IF NEW.stato = 'CANCELLATO' AND OLD.stato != 'CANCELLATO' THEN
        UPDATE prenotazioni
        SET stato_prenotazione = 'CANCELLATA'
        WHERE numero_volo = NEW.numero_volo AND stato_prenotazione != 'CANCELLATA';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_cancella_prenotazioni
AFTER UPDATE OF stato ON voli
FOR EACH ROW EXECUTE FUNCTION propaga_cancellazione_volo();

-- 2. Propagazione dalla Prenotazione ai Ticket
CREATE OR REPLACE FUNCTION libera_posti_prenotazione_cancellata()
RETURNS trigger AS $$
BEGIN
    IF NEW.stato_prenotazione = 'CANCELLATA' AND OLD.stato_prenotazione != 'CANCELLATA' THEN
        DELETE FROM tickets
        WHERE codice_prenotazione = NEW.codice_prenotazione;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_elimina_ticket_cancellati
AFTER UPDATE OF stato_prenotazione ON prenotazioni
FOR EACH ROW EXECUTE FUNCTION libera_posti_prenotazione_cancellata();
```

4.2.5. Implementazione Storico tramite Audit Log

Ogni volta che un ticket passeggero viene eliminato (sia manualmente che a seguito di una cancellazione del volo), i suoi dati vengono salvati automaticamente nella tabella di Audit per ragioni di tracciabilità e assistenza (motivi e argomentazioni anticipate nel capitolo 3.2).

```sql
CREATE OR REPLACE FUNCTION registra_log_canc_ticket()
RETURNS trigger AS $$
BEGIN
    INSERT INTO log_cancellazioni_tickets
        (id_ticket_originale, codice_prenotazione,
            nome_passeggero, cognome_passeggero, motivo)
    VALUES
        (OLD.id_ticket, OLD.codice_prenotazione, OLD.nome,
            OLD.cognome, 'Cancellazione volo/prenotazione');

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_canc_ticket
BEFORE DELETE ON tickets
FOR EACH ROW EXECUTE FUNCTION registra_log_canc_ticket();
```

4.3. Implementazione Viste (Views)

Per ottimizzare le interrogazioni provenienti dal livello applicativo (interfaccia grafica Java) e ridurre la complessità delle operazioni di Join ripetitive, sono state implementate due viste. Queste tabelle virtuali aggregano i dati provenienti da entità diverse, fornendo una "preview" informativa completa e pronta alla consultazione quando necessario.

4.3.1. Vista delle Prenotazioni Complete

La vista ***vw_prenotazioni_complete*** unisce le informazioni riassuntive della prenotazione con i dati anagrafici dell'utente che l'ha effettuata e i dettagli del volo associato.

```sql
CREATE VIEW vw_prenotazioni_complete AS
SELECT
    p.codice_prenotazione, p.email, p.numero_volo, p.stato_prenotazione,
    p.numero_passeggeri, p.data_prenotazione,
    u.username, u.is_admin AS tipo_utente,
    v.compagnia_aerea, v.partenza, v.destinazione, v.data_volo AS data,
    v.orario_previsto, v.stato AS stato_volo, v.ritardo, v.tipo_volo
FROM prenotazioni p
JOIN utenti u ON p.email = u.email
JOIN voli v ON p.numero_volo = v.numero_volo;
```

4.3.2. Vista dei Ticket Completi

In modo analogo, la vista ***vw_tickets_complete*** aggrega i dati del singolo biglietto passeggero, arricchendoli con le informazioni generali della prenotazione e del volo, quindi fondamentale per la generazione della carta d'imbarco o del riepilogo utente.

```sql
CREATE VIEW vw_tickets_complete AS
SELECT
    t.id_ticket, t.nome, t.cognome, t.numero_documento,
    t.data_nascita, t.posto_assegnato, t.codice_prenotazione,
    p.email, p.numero_volo, p.stato_prenotazione,
    v.compagnia_aerea, v.partenza, v.destinazione,
    v.data_volo AS data, v.orario_previsto, v.stato AS stato_volo
FROM tickets t
JOIN prenotazioni p ON t.codice_prenotazione = p.codice_prenotazione
JOIN voli v ON p.numero_volo = v.numero_volo;
```